

PROJECT SPECIFICATIONS

1. Engine & Data Classes

These classes define the "rules" of your world.

CarState (Data Transfer Object)

A lightweight class used to sync data between the server and client.

- **Properties:** `int playerId`, `double x`, `y`, `double velocity`, `double angle`, `int currentLap`.
- **Methods:** `byte[] serialize()` / `static CarState deserialize(byte[])`.

PhysicsEngine

A utility class to keep physics calculations consistent across both Client and Server.

- **Vector2D calculatePosition(Car car, InputState input, double deltaTime):** Calculates the next coordinates.
- **double handleFriction(double speed, boolean isOffTrack):** Returns the new speed. If `isOffTrack` is true, apply a heavy multiplier (e.g., 0.3).
- **boolean isColliding(Shape carBounds, List<Shape> obstacles):** Uses JavaFX `Shape.intersect()` to check for wall hits.

2. Client-Side Classes (JavaFX)

Focused on rendering and capturing user input.

GameClient (Main Entry)

- **void start(Stage stage):** Sets up the JavaFX Scene and the `Canvas`.
- **void connectToServer(String ip, int port):** Initializes the socket connection.

InputHandler

- **void handleKeyPressed(KeyEvent e):** Updates a `Set<KeyCode>` or a boolean array (e.g., `isAccelerating = true`).
- **InputState getCurrentInput():** Packages current keys into a small object to send to the server.

TrackRenderer

- **void drawTrack(GraphicsContext gc, RaceTrack track):** Renders the background, boundaries, and DRS zones.
- **void drawCars(GraphicsContext gc, List<Car> allCars):** Loops through all players and draws their car sprites at the interpolated positions.

3. Server-Side Classes (Java)

The "Source of Truth."

GameServer

- **void listen():** Accepts new player connections and assigns playerIDs.
- **void broadcastState():** Sends the CarState of all players to every connected client (typically 20–60 times per second).

RaceManager

- **void updateRaceLogic():** The server-side game loop.
- **void checkLapProgress(Player p):**
 - Logic: If p.bounds intersects Checkpoints.get(p.nextCheckpoint), increment p.nextCheckpoint.
 - If nextCheckpoint > totalCheckpoints, increment lapCount and reset nextCheckpoint.
- **List<Player> getRankings():** Sorts players by lapCount first, then by their current checkpoint progress.

4. Summary Table of Core Methods

Class	Method	Purpose
Car	applyThrust()	Increases velocity based on Formula 1 acceleration curves.

Car	<code>applySteering()</code>	Rotates the car's angle based on current velocity (faster = wider turns).
Track	<code>getSurfaceType(x, y)</code>	Returns if the coordinate is "Asphalt", "Grass", or "Wall".
Network	<code>sendInput()</code>	Client: Sends movement keys to the Server.
Network	<code>onStateUpdate()</code>	Client: Updates local car positions based on Server data.
UI	<code>updateLeaderboard()</code>	Updates the JavaFX text nodes showing positions (1st, 2nd, etc.).

5. Development Strategy: The "Golden Loop"

1. **Local First:** Build the `Car` and `PhysicsEngine` first. Make sure you can drive a single car around a `Canvas` with smooth collisions.
2. **Add The Server:** Create a simple Server that just echoes back whatever the Client sends.
3. **Authority:** Move the "Lap Counting" and "Collision Validation" to the Server. If the Server says a player hit a wall, it overrides the Client's local position to prevent cheating.
4. **Interpolation:** Since network messages can be jittery, use **Linear Interpolation (Lerp)** on the Client to smoothly move cars between the last known position and the new position received from the server